

Exact, Bounded, or Relaxed: Proving Which Floating-Point Guarantee a Compiler Rewrite Keeps

A Aswanth Raj

School of Computer Science and Engineering, VIT Vellore
Vellore, India

Dr. RANJITHKUMAR S

School of Computer Science and Engineering, VIT Vellore
Vellore, India

Abstract

Compilers offer a binary choice for floating-point code: strict IEEE 754 semantics, or “fast-math” permissions under which reassociation and identity rewrites may change results. The choice conflates three different guarantees. A rewrite may be *bit-identical* because every value it touches is exact; it may change bits while keeping the source program’s worst-case error bound; or it may hold only over the real numbers. We show that a compiler can prove, per rewrite, which of the three it keeps, and can report the weakest one reaching each output as a certificate. The proofs come from declared input domains through a grid-magnitude abstraction: we prove that a matrix chain whose every contiguous sub-chain fits in 53 significand bits is exact under every bracketing, and that the standard componentwise error bound of a chain is identical for all bracketings, which licenses bound-preserving reassociation under a range condition alone. We implement three contracts and per-output certificates in a matrix-language compiler. On 15,000 generated programs, the strict contract recovers 84.5% of the algebraic saving on int8 data with bit-identical output, and the bounded contract all of it on bounded real data. Across 57,600 executions, no output certified bit-identical differed from the unoptimized program; this check found, and we fixed, one unsound certificate in an earlier version. In GCC, the reassociation such a certificate licenses speeds integer-valued reductions by up to 6.54× with identical bits and makes OpenMP reductions reproducible across thread counts. A preregistered study on 2,700 real-valued chains finds the cheaper bracketing more accurate than source order (median error ratio 0.941).

Keywords: floating-point arithmetic, compiler optimization, fast-math, reassociation, abstract interpretation, numerical contracts, matrix chains

1 Introduction

Floating-point addition and multiplication are not associative, and $x \cdot 0 = 0$, $x + 0 = x$ and $x \cdot I = x$ fail for signed zeros, NaN and infinities [1, 2]. Compilers therefore forbid such rewrites by default and offer permissions that enable them: GCC’s `-ffast-math` and its components, or LLVM’s per-instruction fast-math flags [3]. A permission states what the compiler *may* do. It does not state what the result *is*: after `-fassociative-math` a programmer cannot tell which

outputs still equal the strict program, which changed within a known error bound, and which changed arbitrarily.

These are three different guarantees, and a single switch conflates them. Consider the chain $R = ABC$ with $A, C \in \mathbb{R}^{100 \times 2}$ and $B \in \mathbb{R}^{2 \times 100}$. Source order costs 69,800 scalar operations; $A(BC)$ costs 1,396. If A , B and C hold 8-bit integers, every product and partial sum of either bracketing is an integer of at most 29 bits, so both bracketings compute the same bits: the rewrite is not merely safe but *bit-identical*, and a strict compiler should apply it. If the matrices hold real values in $[-1, 1]$, the bits change, yet we show that the textbook componentwise error bound of the chain is the same for every bracketing: the rewrite is *bound-preserving*. Only if values are unbounded, so that one bracketing may overflow where the other does not, is the rewrite merely *relaxed*, valid over the reals and nowhere else.

The difference between these cases is not in the program text but in the values the program will see. We make that knowledge explicit: program inputs declare a value domain (`input(int8)`, `input(real(1))`), the runtime checks each input against its domain before execution, and the compiler derives from the domains facts precise enough to decide, rewrite by rewrite, which guarantee holds. Each output is then certified with the weakest guarantee among the rewrites that reach it.

Contributions.

- A lattice of three numerical contracts (strict, bounded, algebraic) whose proof obligations are per rewrite, and per-output guarantee certificates that compose them, including the inheritance rule that keeps certificates sound when a proof reads facts produced by a weaker rewrite (Section 3).
- A grid-magnitude abstract domain and an exactness theorem for matrix chains: if every contiguous sub-chain fits in 53 significand bits, every bracketing is exact and bit-identical, including the sign of zero (Section 4).
- A proof that the standard componentwise bound of a chain is invariant under bracketing, which reduces bound-preserving reassociation to a range check (Section 5), and a segmented chain-ordering algorithm that applies the cheapest bracketing each contract admits (Section 6).
- An implementation in a compiler with an executable workspace, and an evaluation of recovery, certificate

soundness, threshold tightness, value in a production C compiler, test adequacy by mutation, and a preregistered accuracy study (Section 9).

2 Setting

MatrixLang is a statically shaped, straight-line language over binary64 matrices and scalars. Its compiler (flex, bison, C11) performs shape checking, lowers to three-address code (TAC), optimizes, and executes on a typed stack virtual machine. A matrix product entry is evaluated as $c_{ij} = (\dots((+0 + a_{i1}b_{1j}) + a_{i2}b_{2j}) \dots) + a_{in}b_{nj}$: the accumulator starts at +0, each term is one rounded multiplication and one rounded addition in round-to-nearest, and no fused multiply-add is used (-ffp-contract=off). Outputs are compared in hexadecimal, which distinguishes every binary64 value including -0 and NaN.

The optimizer performs constant folding, common-subexpression elimination, copy propagation and dead-code elimination, which are bit-exact; algebraic identity and zero rewrites ($x \cdot 1$, AI , $x + 0$, $A \cdot 0$, $0 \cdot A$); and matrix-chain reordering by dynamic programming over the modeled cost $mp(2n-1)$ of an $m \times n$ by $n \times p$ product. Before this work a single flag decided all value-changing rewrites at once.

Input domains. A declaration `matrix A[40,2] = input(int8)`; introduces a run-time input whose values the compiler never sees. Domains are `bool`, `uint8`, `int8`, `uint16`, `int16`, `int32`, `int(lo,hi)`, `real` (any finite value), `real(B)` ($|x| \leq B$) and `real(lo,hi)`. Values arrive from files or a seeded generator and are checked before execution: finite, integral for integer domains, within bounds, and with -0 stored as +0 in integer and non-negative domains. The check is what makes domain-derived facts sound; without input declarations every value would be a compile-time constant and constant folding would subsume optimization.

3 Contracts and certificates

Definition 1 (Guarantee levels). *A rewrite of program P into P' is bit-identical if every output of P' equals the corresponding output of P bit for bit, for every input in the declared domains; bound-preserving if every output of P' satisfies the standard-model worst-case error bound of P 's evaluation order and NaN and infinity occur in P' exactly where they occur in P ; and relaxed otherwise, when it is an identity over the reals.*

The levels are ordered bit-identical < bound-preserving < relaxed. A *contract* names the weakest level a rewrite may have and still be applied: *strict* (the default) admits only bit-identical rewrites, *bounded* admits bound-preserving ones, and *algebraic* admits all. Each candidate rewrite computes its level from the facts of its operands; the contract applies or declines it, and a declined rewrite is reported with the contract that would permit it. Constant folding, common

subexpressions, copy propagation, dead-code elimination and double transposition are bit-exact and need no proof.

Certificates. Every TAC instruction carries the weakest level of the rewrites applied to it. The certificate of an output is the weakest level in the dependency cone of its print, together with the reasons recorded by those rewrites. The compiler prints, for example:

Contract: bounded. Every output must keep the source order's worst-case error bound.

1. bit-identical `print(R)`
 - $A * B * C$ is exact: every intermediate of every bracketing needs at most 51 of 53 significand bits
2. bound-preserving `print(S)`
 - $X * B * C$ keeps the source-order error bound: $|error| \leq 2.89e-15 |A| \dots |A_k|$ under every bracketing,
 - and no intermediate can overflow

Inheritance. A dependency cone in the *optimized* program is not enough. In $R = 0 \cdot (1 \cdot (0 \cdot N)^T)$ with $N = -A$, the inner $0 \cdot N$ may be -0; the bounded contract rewrites it to +0 as a bound-preserving rewrite. The outer product is then provably +0, a bit-identical rewrite, but only because of the inner one, and once the outer rewrite drops every use of the inner product, dead-code elimination removes it and its label. Our soundness experiment (Section 9.2) found exactly this case in an earlier version. A rewrite now inherits the weakest level in the dependency cone of the operands its proof read, at the time of the rewrite.

4 Exactness from input domains

The grid-magnitude domain. A fact about a matrix X is a tuple (g, m, fin, nn, nz) : every entry is a multiple of 2^g ($g \geq -1074$), has magnitude at most m , is finite, is non-negative, and is not -0, respectively. Integer domains give $g = 0$ and $m = \max(|lo|, |hi|)$; bounded reals give $g = -1074$ (every binary64 value is a multiple of 2^{-1074}) and $m = B$; unbounded reals give $m = \text{DBL_MAX}$. Literals give exact facts. Transfer functions use upward-rounded bound arithmetic: a sum takes the smaller grid and adds magnitudes; a product of entries adds grids; a matrix product of inner dimension n has grid $g_a + g_b$ and magnitude $n m_a m_b$, inflated by $(1 + 2nu)$ when not provably exact. Non-finite or overflowing results lose every fact. A matrix-product entry is never -0, because its accumulator starts at +0.

Lemma 4.1 (Rounding preserves grids). *If x is a real multiple of 2^g with $g \geq -1074$ and $|x| \leq \text{DBL_MAX}$, then $\text{fl}(x)$ is a multiple of 2^g , and $\text{fl}(x) = x$ when $|x| \leq 2^{g+53}$.*

Proof. Let $e = \lfloor \log_2 |x| \rfloor$; the spacing of binary64 near x is $2^{\max(e-52, -1074)}$. If $|x| < 2^{g+53}$ then $e - 52 \leq g$, so the spacing divides 2^g and x is representable. If $|x| = 2^{g+53}$, x is a power of two in range. Otherwise $e \geq g + 53$, the spacing 2^{e-52} is

a multiple of 2^g , and so is every representable neighbor of x . $\square$

Let $A_1 \cdots A_k$ be a chain with $A_l \in \mathbb{R}^{d_{l-1} \times d_l}$ and facts (g_l, m_l) . For $i \leq j$ let $G_{ij} = \sum_{l=i}^j g_l$ and $M_{ij} = \prod_{l=i}^j m_l \cdot \prod_{l=i}^{j-1} d_l$.

Theorem 4.2 (Exact chains). *If every operand is finite and, for all $1 \leq i \leq j \leq k$, either $M_{ij} = 0$ or $G_{ij} \geq -1074$ and $M_{ij} \leq 2^{G_{ij}+53}$, then every bracketing of $A_1 \cdots A_k$, evaluated in the VM's order, rounds no operation, computes the exact product, and produces no -0 entry. All bracketings therefore produce bit-identical results.*

Proof. By induction on the bracketing tree, every node T for the sub-chain $i..j$ computes exactly $P_{ij} = A_i \cdots A_j$, whose entries are bounded by M_{ij} and are multiples of $2^{G_{ij}}$. Leaves hold by assumption. At a node $T = LR$ splitting after s , the children are exact by hypothesis. Every scalar product $\ell_{ab} r_{bc}$ is a multiple of $2^{G_{is}+G_{s+1,j}} = 2^{G_{ij}}$, and every partial sum of the d_s terms has magnitude at most $d_s M_{is} M_{s+1,j} = M_{ij} \leq 2^{G_{ij}+53}$. By Lemma 4.1 each product and each addition is exact. If $M_{ij} = 0$, some operand of $i..j$ is zero, all products are exact zeros of finite factors, and the result is 0. Finally, in round-to-nearest a sum is -0 only if both addends are -0 ; an accumulator starting at $+0$ is therefore never -0 , and neither is any entry of any bracketing. Exact values with the same sign convention for zero have the same bits. $\square$

The condition must hold for every contiguous sub-chain, not only the whole chain. If the last operand is a zero matrix, $M_{1k} = 0$, yet $A_1 A_2$ may overflow to ∞ and $\infty \cdot 0 = \text{NaN}$ in source order, while the reordered chain computes 0; our mutation study contains this witness (Section 9.5). For $\text{int}(\emptyset, m)$ chains of shape $100 \times 2 \cdot 2 \times 100 \cdot 100 \times 2$ the largest admissible bound is $m^* = 35,578$, since $200 \cdot 35578^3 \leq 2^{53} < 200 \cdot 35579^3$.

5 Bound-preserving reassociation

In the standard model $\text{fl}(a \circ b) = (a \circ b)(1 + \delta)$, $|\delta| \leq u = 2^{-53}$, absent underflow and overflow, an inner product of length n evaluated in any order satisfies $|\text{fl}(x^T y) - x^T y| \leq \gamma_n |x|^T |y|$ with $\gamma_n = nu / (1 - nu)$ [4].

Theorem 5.1 (Bracketing invariance). *For every bracketing T of $A_1 \cdots A_k$ evaluated without underflow or overflow,*

$$|\hat{T} - A_1 \cdots A_k| \leq \left(\prod_{l=1}^{k-1} (1 + \gamma_{d_l}) - 1 \right) |A_1| \cdots |A_k|$$

componentwise, where $d_1, \dots, d_{k-1}$ are the inner dimensions.

Proof. Write $\Pi_T = \prod (1 + \gamma_d)$ over the dimensions contracted inside T and $|A|_T$ for the product of the operands' absolute values. We show $|\hat{T} - P_T| \leq (\Pi_T - 1) |A|_T$ by induction. For $T = LR$ contracting d , let $\hat{L} = P_L + E_L$ and $\hat{R} = P_R + E_R$ with $|E| \leq (\Pi_L - 1) |A|_L$, so $|\hat{L}| \leq \Pi_L |A|_L$. Then $\hat{T} = \hat{L} \hat{R} + F$ with

$$|F| \leq \gamma_d |\hat{L}| |\hat{R}| \text{ and } \hat{L} \hat{R} - P_L P_R = E_L P_R + P_L E_R + E_L E_R. \text{ Using } |P| \leq |A|,$$

$$|\hat{T} - P_T| \leq [(\Pi_L - 1) + (\Pi_R - 1) + (\Pi_L - 1)(\Pi_R - 1) + \gamma_d \Pi_L \Pi_R] |A|_T = (\Pi_L \Pi_R (1 + \gamma_d) - 1) |A|_T.$$

Every inner dimension is contracted at exactly one node of any bracketing, so $\Pi_T = \prod_l (1 + \gamma_{d_l})$ for every T . $\square$

The bound coefficient does not depend on the bracketing, so a rewrite between bracketings preserves it whenever both are evaluated without overflow, and NaN and infinity cannot arise in either. The bounded contract therefore admits a reordering when every sub-chain's magnitude bound stays below 2^{1020} (leaving slack for the Π factors). This is why `real(1)` chains are reordered under the bounded contract while chains over unbounded `real` are not: there no fact excludes overflow. Theorem 5.1 also explains why an earlier attempt to trade cost against worst-case accuracy in the chain dynamic program was degenerate: all bracketings tie. Underflow adds an absolute term to the standard model; we do not claim that term is bracketing-invariant (Section 11).

6 Segmented chain ordering

A contract may admit a cheaper bracketing for part of a chain but not the whole. For a chain $A_1 \cdots A_k$ and source-order prefixes $P_i = A_1 \cdots A_i$ (computed left to right, as the source does), the pass computes

$$\text{best}[j] = \min_{i < j} \text{best}[i] + \text{cost}(P_i \cdot A_{i+1} \cdots A_j),$$

where a segment of two operands is the source step, always legal, and a longer segment may use its cheapest bracketing (the classical $O(k^3)$ program [5]) if its level is admitted: exact by Theorem 4.2 for strict, exact or range-safe for bounded, anything for algebraic. Segment facts use the facts of P_i as their first operand. Ties prefer the stronger guarantee, so an exact tree is chosen over a bound-preserving tree of equal cost. Only segment roots carry a guarantee: interior products have no counterpart in the source program, and labelling them let common-subexpression elimination spread one chain's proof to an unrelated output. On a four-operand $\text{int}(\emptyset, m)$ chain $24 \times 3 \cdot 3 \times 24 \cdot 24 \times 3 \cdot 3 \times 24$, the whole chain is exact only for $m \leq 2,541$, but its first three operands are exact up to $m = 50,013$, and the strict contract reorders that segment (Section 9.3).

7 IEEE side conditions of identity rewrites

Table 1 lists the conditions. A matrix product with I is not the identity in IEEE 754: each entry is $+0 + a \cdot 1 + b \cdot 0 + \dots$, which maps -0 to $+0$ and ∞ to NaN through $\infty \cdot 0$. Adding $+0$ maps -0 to $+0$. A scaling takes the sign of each factor, so $0 \cdot (-3) = -0$. The rule $0 - x \rightarrow -x$ is absent: it maps $+0 - (+0) = +0$ to -0 and saves no arithmetic.

Table 1. Levels of identity and zero rewrites from operand facts. “fin” is finite, “nz” has no -0 entry, “nn” is non-negative.

Rewrite	bit-identical if	else
$x \cdot 1 \rightarrow x$ (scalar unit)	always	–
$AI, IA \rightarrow A$ (product)	fin, nz	bounded if fin
$x + 0, 0 + x \rightarrow x$	x nz	bounded
$x - 0 \rightarrow x$	x nz or zero nz	bounded
$A \cdot 0 \rightarrow 0$ (product)	other operand fin	relaxed
$0 \cdot A \rightarrow 0$ (scaling)	both nn and fin	bounded if fin

8 Implementation

The compiler is 6,784 lines of C11, flex and bison. The fact domain and the segmented chain pass take 1,193 of them (facts.c, chain.c); input domains and their load-time checks take 555 (domain.c, inputs.c); contracts, side conditions and certificates are part of optimize.c. The command line selects `-fp-strict` (default), `-fp-bounded` or `-fp-algebraic`; `-no-proofs` disables facts and reproduces the previous strict optimizer; `-certificate` prints per-output guarantees; and `-random-inputs seed` draws inputs reproducibly. A browser workspace runs the same compiler locally, shows the three contracts and each output’s certificate, checks every certificate against an unoptimized run, and presents the measurements of Section 9.

9 Evaluation

We ask: (RQ1) how much of the algebraic optimization each contract recovers with proofs; (RQ2) whether certificates are sound; (RQ3) how close the exactness threshold is to the true boundary; (RQ4) what a certificate is worth to a production compiler; (RQ5) which tests notice a missing side condition; and (RQ6) whether cheaper bracketings are more accurate. All scripts, seeds and raw observations are in the artifact (make contracts); every number below is generated from them. Experiments ran on an Intel Core Ultra 9 275HX (24 cores, AVX2) under Windows 11 with GCC 15.2.

Workloads. No benchmark suite or dataset could be downloaded for this study, so workloads are generated: straight-line programs whose matrices are inputs over one value domain (bool to int32, real(1), unbounded real, or a per-matrix mixture), with six shape profiles (heterogeneous, square, narrow, balanced, elementwise, and an algebraic profile that adds identity, zero, negation and negative-scaling expressions). Two literal populations reproduce an earlier study: small integers (“dyadic”) and values spread over $10^{-12}..10^{12}$ (“broad”).

9.1 RQ1: Recovery

Table 2 and Figure 1 report 60,000 compilations with no failures. Without proofs, strict optimization removes almost nothing. With proofs it recovers 98.8% of the algebraic saving

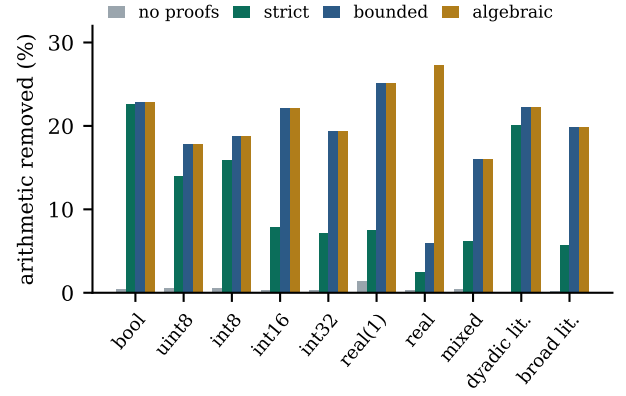


Figure 1. RQ1. Modeled arithmetic removed per value domain and contract.

on bool, 84.5% on int8 and 77.7% on uint8 while keeping every output bit-identical; on int16 and int32 the 53-bit budget is exhausted by longer chains and recovery falls to 34.6% and 35.9%, mostly from identity and zero rewrites. The bounded contract recovers the entire saving on every domain with a magnitude bound, and 21.2% on unbounded real, where it cannot exclude overflow. On the earlier study’s dyadic literals the strict contract recovers 90.1%: those programs were exact all along, which explains why an earlier algebraic mode never changed their output. How far the proofs reach depends on the domain’s width. Recompiling 200 heterogeneous and narrow programs with every matrix over $\text{int}(-(2^b - 1), 2^b - 1)$, the strict contract proves exact 99% of the chain reorderings the algebraic contract applies at $b = 3$, 88% at $b = 7$, 45% at $b = 15$, 2% at $b = 17$, and none from $b = 19$: a product of three such operands already needs more than 53 bits. The proofs are cheap: compiled one at a time, a sample of 300 of these programs took a median 8.0 ms end to end under strict against 8.0 ms without proofs (median ratio 1.00, largest 1.12), a time dominated by process start-up.

9.2 RQ2: Soundness of certificates

Each program ran unoptimized and under every contract on three seeded input draws, 57,600 executions in all (Table 3). Strict output equalled unoptimized output on all 57,600 outputs. Under the bounded and algebraic contracts, 0 and 0 outputs certified bit-identical differed; the outputs that did differ were all certified bound-preserving or relaxed. This experiment is what found the inheritance defect of Section 3: before the fix, 12 outputs per contract, from four programs of the algebraic profile, were certified bit-identical and differed. Execution checks a certificate for drawn inputs only; the certificate itself is a claim over the whole domain, which Theorem 4.2 supports. Certificates are also conservative: of 3,964

Table 2. RQ1. Modeled arithmetic removed under each contract (pooled over 15,000 programs, ten seeds, six profiles), and recovery: the share of the algebraic contract’s saving obtained over the unproved strict optimizer.

Domain	Modeled arithmetic removed (%)				Recovery (%)	
	no proofs	strict	bounded	algebraic	strict	bounded
bool	0.4	22.6	22.8	22.8	98.8	100.0
uint8	0.5	13.9	17.8	17.8	77.7	100.0
int8	0.5	15.9	18.7	18.7	84.5	100.0
int16	0.3	7.8	22.1	22.1	34.6	100.0
int32	0.3	7.1	19.4	19.4	35.9	100.0
real(1)	1.4	7.5	25.1	25.1	26.0	100.0
real	0.2	2.5	6.0	27.2	8.4	21.2
mixed	0.4	6.2	16.0	16.0	37.1	100.0
dyadic literals	0.1	20.1	22.3	22.3	90.1	100.0
broad literals	0.2	5.7	19.8	19.8	28.1	100.0

Table 3. RQ2. Outputs of 4,800 executed programs on three input seeds, compared bit for bit with the unoptimized program.

Contract	Outputs	Differ	Bit-identical		Bound-preserving	
			certified	differ	certified	differ
strict	57,600	0	57,600	0	0	0
bounded	57,600	9,160	45,708	0	11,892	9,160
algebraic	57,600	10,763	43,920	0	11,787	9,160

outputs the bounded contract certified below bit-identical, 853 (21.5%) printed the unoptimized bits on all three draws; the certificate bounds what may change, not what will.

9.3 RQ3: Tightness of the exactness threshold

For four chain families we found the largest m for which strict reorders $\text{int}(\emptyset, m)$ data by binary search on the compiler’s certificates, then ran seventeen witness inputs at 14 values of m around it under a contract that always reorders (Table 4). The compiler’s threshold equals the closed form of Theorem 4.2 in every family; for the four-operand chain it is the threshold of its first segment. None of 952 strict runs differed from unoptimized output. For the four-operand chain a witness exists at $m^* + 1$, so the certificate is tight there. For three-operand chains, no witness appeared below about $1.1m^*$: the magnitude bound assumes every partial sum reaches its maximum, which these inputs did not produce, so the certificate is conservative by up to ten percent of the domain there.

9.4 RQ4: Value in a production compiler

A certificate is useful beyond our compiler if it licenses something a production compiler would otherwise refuse. We compiled six reductions twice with GCC: strict, and with `-fassociative-math`, `-fno-signed-zeros` and `-fno-trapping-math`, which let GCC reassociate and vectorize them (Table 5). On

certified data both builds printed identical bits, and the licensed build was faster by $1.23\times$ (an `int16` dot product, memory-bound) to $6.54\times$ (an `int8` dot product in `binary32`, compute-bound). The real-valued control changed bits for a comparable speedup. A run-time domain check over both inputs of the `int16` dot product, for data that arrives untyped, costs 95.1% of the strict kernel; MatrixLang performs it once at load time. With OpenMP, whose reduction combines partial sums in an unspecified order, the certified dot product gave one result for every thread count from 1 to 24 and every repetition, while the real-valued one gave thirteen distinct results.

9.5 RQ5: Test adequacy by mutation

Nine mutants each drop one side condition (Table 6). Small-integer programs, the population an earlier differential study relied on, kill no test of mutant 1, which reorders without any proof: by Theorem 4.2 those programs are exact under every bracketing, so the population is structurally blind to the defect. Broad-valued literals kill it in most programs. Side conditions on `-0` and non-finite values are rarely exercised by random programs; one witness program per condition, written from Table 1, kills mutants 1, 2, 4, 6, 7, 8 and 9. Mutant 5 is equivalent: a non-finite fact never carries “no `-0`”, so the `-0` condition already blocks every rewrite it would admit. Mutant 3, which accepts 54 bits, survived every population including all-maximum inputs, consistent with the slack RQ3 measured near the threshold.

9.6 RQ6: Accuracy of cheaper bracketings

Theorem 5.1 says the cheaper bracketing has the same worst-case bound; it says nothing about typical error. We preregistered a study before running it: chains of three to five real-valued operands with dimensions up to 200, values uniform in $[-1, 1]$, uniform in $[0, 1]$, or broad; ten new seeds; error $|\hat{c}_{ij} - c_{ij}| / (|A_1| \cdots |A_k|)_{ij} / u$ against an exact integer reference; and a numpy emulation of the VM’s evaluation

Table 4. RQ3. Threshold found by binary search on the compiler’s certificates versus the closed form, and witnesses on which reordering changes the printed bits (all-maximum, six near-maximum and ten seeded inputs per m).

Chain	m^* (compiler)	closed form	first witness	witnessed m in $(m^*, 1.1m^*]$	strict differ
100x2	35,578	35,578	$1.1000m^*$	1/6	0
24x3x4	50,013	50,013	$1.0000m^*$	6/6	0
40x8	30,419	30,419	$1.1000m^*$	1/6	0
50x4	35,578	35,578	$1.1000m^*$	1/6	0

Table 5. RQ4. C kernels on integer-valued binary64 (binary32 for fdot), sized so every partial sum is exact, compiled by GCC 15.2 -O3 -march=native strict and with reassociation licensed. Medians of 7 processes $\times$ 21 repetitions, one thread.

Kernel	strict (μ s)	licensed (μ s)	speedup	same bits
dot_i16	414.6	337.8	1.23 $\times$	yes
sum_i32	830.9	337.8	2.46 $\times$	yes
matvec_u8_i16	2,504.4	1,460.3	1.71 $\times$	yes
gemm_i8	4,095.1	1,259.3	3.25 $\times$	yes
fdot_i8	91.6	14.0	6.54 $\times$	yes
dot_real	412.4	341.5	1.21 $\times$	no

order, which matched `matrixc` bit for bit on 120 of 120 validation bracketings. The prediction was that the pooled sign test rejects equality at 0.01 in favour of the cost-optimal order with the same direction in every profile, and that an equal-cost ablation removes the effect.

The cost-optimal bracketing was more accurate in 728 of 1,222 chains with unequal error (Holm $p = 4.6 \times 10^{-11}$; median ratio 0.941, 95% CI [0.925, 0.955]), significantly on heterogeneous shapes (411 of 635, ratio 0.892) and in the same direction but not significantly on narrow shapes (317 of 587, $p = 0.058$), where cheaper bracketings save less. Pairs of distinct bracketings with equal modeled cost showed no difference (652 of 1,296, $p = 0.85$), as an operation-count account predicts. Because equal-cost pairs almost always also have equal intermediate sizes, the ablation does not separate that account from the rival that smaller intermediates cause the effect; we report the association, not the mechanism. No chain exceeded the bound of Theorem 5.1; the largest error reached 0.491 of it.

10 Related work

Fast-math semantics. LLVM attaches fast-math permissions to individual instructions (reassoc, nsz, nnan, ninf and others) [3]; permissions are finer than a global switch but still state what may change, not what holds. Icing gives fast-math style optimizations a semantics in a verified compiler, with optimizations allowed where the program permits them and error analysis on top [6]. Our certificates are derived, not granted: a rewrite’s level follows from input domains,

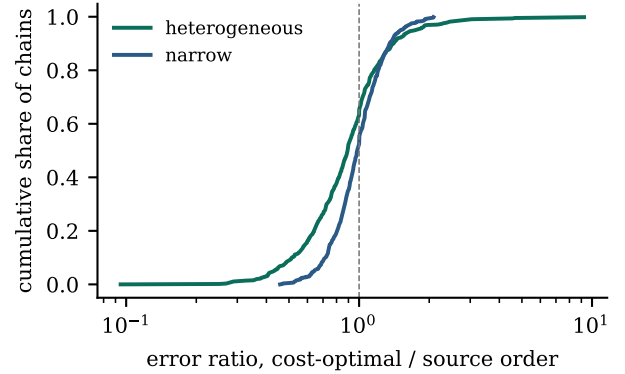


Figure 2. RQ6. Distribution of the error ratio (cost-optimal / source) over chains whose cheaper bracketing differs from source order.

and a strict program gets reassociation where exactness is proved. Alive and Alive-FP verify peephole optimizations, including floating-point preconditions on signed zeros and non-finite values [7, 8]; Alive-Infer infers such preconditions [9]. Our side conditions (Table 1) are of this kind; our contribution is their organisation into guarantee levels and their discharge from domain facts. Boldo et al. verify that CompCert preserves floating-point semantics exactly [10]; Monniaux catalogues why compiled floating-point code is hard to reason about [11].

Accuracy analysis and repair. Rosa and Daisy compile real-valued specifications to finite precision under error bounds [12, 13]; FPTaylor bounds round-off of expressions [14]; Herbie rewrites expressions for accuracy [15]. These tools bound or improve error for chosen precision; we decide, for a fixed precision, which compiler rewrites keep bits or keep the source program’s bound, and certify each output.

Exactness and reproducibility. Exact integer linear algebra over floating-point units bounds intermediates below 2^{53} , for example to choose recursion depths in FFLAS-FFPACK [16]; error-free transformations make sums exact or accurate [17]; reproducible summation makes parallel reductions order-independent at a cost [18]. Bitwidth analysis computes value ranges to size hardware [19]. Theorem 4.2

Table 6. RQ5. Tests that kill each mutant (number of tests per population in parentheses). A test kills a mutant when the mutant’s strict-optimized output differs from the correct unoptimized output.

Mutant	dyadic (90)	broad (90)	domain (720)	extreme (720)	fixtures (15)	witness (7)
1 reorder without proof	0	51	246	95	2	2
2 whole chain only	0	0	0	0	0	1
3 54-bit threshold	0	0	0	0	0	0
4 AI: ignore -0	3	0	9	22	0	1
5 AI: ignore non-finite	0	0	0	0	0	0
6 A + 0: ignore -0	1	1	6	21	1	2
7 A0: ignore non-finite	0	0	0	3	0	2
8 0A: ignore sign	4	7	45	47	1	1
9 negation yields no -0	4	1	15	20	0	3

Table 7. RQ6. Paired comparison of the cost-optimal and source bracketings on real-valued chains, and the equal-cost ablation. Sign tests are two-sided and exact; Holm-adjusted over the three confirmatory tests; median ratio of errors (cost-optimal / source) with seed-cluster bootstrap 95% intervals.

Population	chains	unequal	cheaper better	p (Holm)	median ratio [95% CI]
heterogeneous	648	635	411	3.1×10^{-13}	0.892 [0.863, 0.913]
narrow	606	587	317	0.058	0.979 [0.954, 1.000]
pooled	1,254	1,222	728	4.6×10^{-11}	0.941 [0.925, 0.955]
<i>Equal-cost ablation: first = fewer intermediate entries, or enumeration order</i>					
heterogeneous	64	59	27	0.6	
narrow	363	352	199	0.016	
square	900	885	426	0.28	
pooled	1,327	1,296	652	0.85	

applies this kind of reasoning as a compiler legality criterion over chains of any bracketing, with signed zeros and sub-chains, and RQ4 shows that it yields reproducibility for free where it applies. The bracketing invariance of Theorem 5.1 follows from the standard analysis of inner products [4]; we claim its use as a contract, not the inequality.

Matrix chains and testing. Optimal chain ordering is classical [5]; Linnea generates linear-algebra code with property-aware cost models [20]. Neither certifies numerical guarantees per output. Csmith-style differential testing [21] and mutation analysis [22] are established; RQ5 shows a structural reason why value-blind differential testing misses reassociation defects.

11 Limitations

MatrixLang is straight-line and statically shaped; loops would need the facts to reach a fixed point, which the lattice permits but we have not implemented. The bounded contract promises the standard-model bound; with gradual underflow both orders need an absolute error term, which we do not show to be invariant. Grids of real-valued data are 2^{-1074} , so exactness proofs apply to integer and dyadic data, not to general reals. Domains are trusted declarations checked

at load time; a domain that is wider than the data loses opportunities, never soundness. Workloads are generated, not drawn from applications, because no dataset or benchmark suite was available offline; RQ4’s speedups come from hand-written kernels compiled by one compiler on one machine. The equal-cost ablation in RQ6 cannot discriminate between two mechanisms. All evaluation shares one front end and one VM with the optimizer; RQ6 validates its emulation against the VM but not against an independent implementation of IEEE arithmetic.

12 Conclusion

A floating-point rewrite keeps one of three guarantees, and which one is often provable from the values a program will see. Declared input domains, a grid-magnitude analysis and two theorems let a compiler apply reassociation under strict semantics where it is exact, under a bounded contract where the error bound is unchanged, and report the guarantee of every output. Certificates checked against execution caught a real defect, and the exactness they establish is worth up to a 6.54× speedup and reproducible parallel reductions to a production compiler.

Data availability

The compiler, experiment scripts, raw observations and workspace are in the artifact repository. `make contracts` repeats every measurement and `make contracts-analyze` regenerates every number, table and figure.

References

- [1] David Goldberg. 1991. What Every Computer Scientist Should Know About Floating-Point Arithmetic. *ACM Comput. Surv.* 23, 1 (1991), 5–48. <https://doi.org/10.1145/103162.103163>
- [2] IEEE. 2019. *IEEE Standard for Floating-Point Arithmetic*. IEEE Std 754-2019.
- [3] LLVM Project. LLVM Language Reference Manual: Fast-Math Flags. <https://llvm.org/docs/LangRef.html#fast-math-flags>
- [4] Nicholas J. Higham. 2002. *Accuracy and Stability of Numerical Algorithms* (2nd ed.). SIAM.
- [5] T. C. Hu and M. T. Shing. 1982. Computation of Matrix Chain Products. Part I. *SIAM J. Comput.* 11, 2 (1982), 362–373.
- [6] Heiko Becker, Eva Darulova, Magnus O. Myreen, and Zachary Tatlock. 2019. Icing: Supporting Fast-Math Style Optimizations in a Verified Compiler. In *CAV 2019*. https://doi.org/10.1007/978-3-030-25543-5_10
- [7] Nuno P. Lopes, David Menendez, Santosh Nagarakatte, and John Regehr. 2015. Provably Correct Peephole Optimizations with Alive. In *PLDI*.
- [8] David Menendez, Santosh Nagarakatte, and Aarti Gupta. 2016. Alive-FP: Automated Verification of Floating Point Based Peephole Optimizations in LLVM. In *SAS*.
- [9] David Menendez and Santosh Nagarakatte. 2017. Alive-Infer: Data-Driven Precondition Inference for Peephole Optimizations in LLVM. In *PLDI*.
- [10] Sylvie Boldo, Jacques-Henri Jourdan, Xavier Leroy, and Guillaume Melquiond. 2015. Verified Compilation of Floating-Point Computations. *J. Autom. Reason.* 54, 2 (2015), 135–163.
- [11] David Monniaux. 2008. The Pitfalls of Verifying Floating-Point Computations. *ACM Trans. Program. Lang. Syst.* 30, 3 (2008).
- [12] Eva Darulova and Viktor Kuncak. 2014. Sound Compilation of Reals. In *POPL*.
- [13] Eva Darulova, Anastasiia Izycheva, Fariha Nasir, Fabian Ritter, Heiko Becker, and Robert Bastian. 2018. Daisy – Framework for Analysis and Optimization of Numerical Programs. In *TACAS*.
- [14] Alexey Solov'yev, Charles Jacobsen, Zvonimir Rakamarić, and Ganesh Gopalakrishnan. 2015. Rigorous Estimation of Floating-Point Round-off Errors with Symbolic Taylor Expansions. In *FM*.
- [15] Pavel Panchekha, Alex Sanchez-Stern, James R. Wilcox, and Zachary Tatlock. 2015. Automatically Improving Accuracy for Floating Point Expressions. In *PLDI*.
- [16] Jean-Guillaume Dumas, Pascal Giorgi, and Clément Pernet. 2008. Dense Linear Algebra over Word-Size Prime Fields: the FFLAS and FFPACK Packages. *ACM Trans. Math. Softw.* 35, 3 (2008).
- [17] Takeshi Ogita, Siegfried M. Rump, and Shin'ichi Oishi. 2005. Accurate Sum and Dot Product. *SIAM J. Sci. Comput.* 26, 6 (2005).
- [18] James Demmel and Hong Diep Nguyen. 2013. Fast Reproducible Floating-Point Summation. In *ARITH*.
- [19] Mark Stephenson, Jonathan Babb, and Saman Amarasinghe. 2000. Bitwidth Analysis with Application to Silicon Compilation. In *PLDI*.
- [20] Henrik Barthels, Christos Psarras, and Paolo Bientinesi. 2021. Linnea: Automatic Generation of Efficient Linear Algebra Programs. *ACM Trans. Math. Softw.* 47, 3 (2021). <https://doi.org/10.1145/3446632>
- [21] Xuejun Yang, Yang Chen, Eric Eide, and John Regehr. 2011. Finding and Understanding Bugs in C Compilers. In *PLDI*. <https://doi.org/10.1145/1993316.1993532>
- [22] Yue Jia and Mark Harman. 2011. An Analysis and Survey of the Development of Mutation Testing. *IEEE Trans. Softw. Eng.* 37, 5 (2011).